

2975. Maximum Square Area by Removing Fences From a Field

Solved 

Medium

 Topics

 Companies

 Hint

There is a large $(m - 1) \times (n - 1)$ rectangular field with corners at $(1, 1)$ and (m, n) containing some horizontal and vertical fences given in arrays `hFences` and `vFences` respectively.

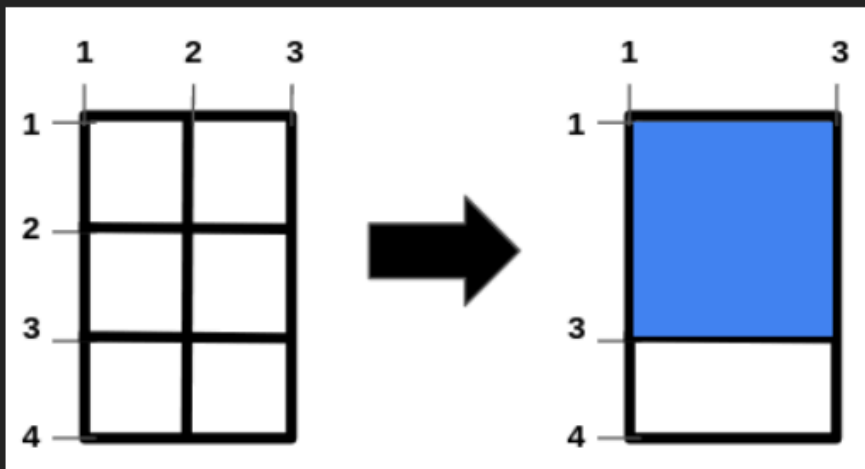
Horizontal fences are from the coordinates $(hFences[i], 1)$ to $(hFences[i], n)$ and vertical fences are from the coordinates $(1, vFences[i])$ to $(m, vFences[i])$.

Return the **maximum** area of a **square** field that can be formed by **removing** some fences (**possibly none**) or -1 if it is impossible to make a square field.

Since the answer may be large, return it **modulo** $10^9 + 7$.

Note: The field is surrounded by two horizontal fences from the coordinates $(1, 1)$ to $(1, n)$ and $(m, 1)$ to (m, n) and two vertical fences from the coordinates $(1, 1)$ to $(m, 1)$ and $(1, n)$ to (m, n) . These fences **cannot** be removed.

Example 1:



Input: `m = 4, n = 3, hFences = [2,3], vFences = [2]`

Output: `4`

Explanation: Removing the horizontal fence at 2 and the vertical fence at 2 will give a square field of area 4.

Example 2:



Input: $m = 6$, $n = 7$, $hFences = [2]$, $vFences = [4]$

Output: -1

Explanation: It can be proved that there is no way to create a square field by removing fences.

Constraints:

- $3 \leq m, n \leq 10^9$
- $1 \leq hFences.length, vFences.length \leq 600$
- $1 < hFences[i] < m$
- $1 < vFences[i] < n$
- $hFences$ and $vFences$ are unique.

Python:

class Solution:

```
def maximizeSquareArea(self, m: int, n: int, hFences: List[int], vFences: List[int]) -> int:
    hFences.extend([1,m])
    vFences.extend([1,n])
    hFences.sort()
    vFences.sort()
    h,v = len(hFences),len(vFences)
    seen_h=set()
```

```

for i in range(h-1):
    for j in range(i+1,h):
        seen_h.add(hFences[j]-hFences[i])
m=-1
for i in range(v-1):
    for j in range(i+1,v):
        q = vFences[j]-vFences[i]
        if q in seen_h:
            m = max(m,q)
MOD = 10**9+7
return m**2 % MOD if m>0 else -1

```

JavaScript:

```

var maximizeSquareArea = function(totalHeight, totalWidth, horizontalCuts, verticalCuts) {
    const MOD = 1000000007n;

    const prep = (cuts, limit) => [1, ...cuts.sort((a,b)=>a-b), limit];

    const h = prep(horizontalCuts, totalHeight);
    const v = prep(verticalCuts, totalWidth);

    const set = new Set();

    for (let i=0; i<h.length; i++)
        for (let j=i+1; j<h.length; j++)
            set.add(h[j]-h[i]);

    let best = 0;

    for (let i=0; i<v.length; i++)
        for (let j=i+1; j<v.length; j++) {
            const d = v[j]-v[i];
            if (d > best && set.has(d)) best = d;
        }

    if (best === 0) return -1;

    return Number((BigInt(best) * BigInt(best)) % MOD);
};

```

Java:

```

class Solution {
    public int maximizeSquareArea(int m, int n, int[] hFences, int[] vFences) {
        // 1. Add implicit boundary fences to new lists
        // We use Arrays to facilitate sorting and resizing
    }
}

```

```

int[] h = Arrays.copyOf(hFences, hFences.length + 2);
h[hFences.length] = 1;
h[hFences.length + 1] = m;

int[] v = Arrays.copyOf(vFences, vFences.length + 2);
v[vFences.length] = 1;
v[vFences.length + 1] = n;

// 2. Sort arrays
Arrays.sort(h);
Arrays.sort(v);

// 3. Store all possible horizontal gaps
Set<Integer> hGaps = new HashSet<>();
for (int i = 0; i < h.length; i++) {
    for (int j = i + 1; j < h.length; j++) {
        hGaps.add(h[j] - h[i]);
    }
}

// 4. Check vertical gaps
long maxSide = -1;
for (int i = 0; i < v.length; i++) {
    for (int j = i + 1; j < v.length; j++) {
        int currentGap = v[j] - v[i];
        if (hGaps.contains(currentGap)) {
            maxSide = Math.max(maxSide, currentGap);
        }
    }
}

if (maxSide == -1) return -1;

// 5. Return result modulo 10^9 + 7
long MOD = 1_000_000_007;
return (int) ((maxSide * maxSide) % MOD);
}
}

```